Week 1: Introduction to Computing & Core Concepts

Topics:

- What is Computing?
- Components of a Computer System (Hardware & Software)
- Data vs Information
- Input–Process–Output (IPO) Model
- Introduction to Computational Thinking

Learning Outcomes:

Students should be able to:

- Explain what computing is
- Describe IPO model
- Identify hardware and software components

Practical:

- Draw IPO diagram for:
    - ATM machine
    - Result processing system
    - Online shopping system

✅ **Week 2: Problems and Types of Problems**

Topics:

- What is a Problem?
- Characteristics of a Good Problem Definition
- Routine Problems
- Non-Routine Problems
- Well-defined vs Ill-defined problems

Learning Outcomes:

Students should be able to:

- Identify different types of problems
- Differentiate routine and non-routine problems

Practical:

Classify the following:

- Calculating student GPA
- Designing traffic control system
- Predicting stock prices
- Finding the largest of three numbers

✅ Week 3: Introduction to Algorithms & Heuristics

Topics:

- Meaning of Algorithm
- Characteristics of a Good Algorithm
- Examples of Algorithms (step-by-step daily tasks)
- What are Heuristics?
- Difference between Algorithm and Heuristic

Learning Outcomes:

Students should be able to:

- Write simple algorithms in steps
- Explain heuristics

Practical:

Write algorithm for:

- Finding the largest of three numbers
- Converting Celsius to Fahrenheit
- Login authentication process

✅ Week 4: Solvable and Unsolvable Problems

Topics:

- What makes a problem solvable?
- Decidability concept (basic introduction)
- Example: Halting Problem (conceptual only)

- Computational limits
- Efficiency (basic idea of time & space)

Learning Outcomes:

Students should:

- Understand limitations of computing
- Identify problems that are practically unsolvable

Practical:

Discussion-based:

- Can every problem be solved by a computer?
- Why is password cracking difficult?

✅ Week 5: Problem-Solving Techniques I

Topics:

- Abstraction
- Analogy
- Brainstorming
- Trial and Error
- Hypothesis Testing
- Research

Learning Outcomes:

Students should:

- Apply appropriate technique to different problems

Activity:

Group activity:

- Solve “How to reduce traffic congestion in Abuja?”
- Identify which techniques were used

✅ Week 6: Problem-Solving Techniques II

Topics:

- Reduction
- Literal Thinking
- Means-End Analysis
- Method of Focal Object
- Morphological Analysis
- Root Cause Analysis
- Proof
- Divide and Conquer

Learning Outcomes:

Students should:

- Apply divide and conquer in simple problems
- Use root cause analysis

Practical:

- Break down “Student Result Management System”
- Use divide and conquer to design it

✅ Week 7: Solution Formulation & Design Tools

Topics:

- General Problem-Solving Process
    1. Problem Identification
    2. Analysis
    3. Design
    4. Implementation
    5. Testing
    6. Evaluation & Refinement
- Flowcharts (symbols and drawing)
- Pseudocode
- Decision Tables
- Decision Trees

Learning Outcomes:

Students should:

- Draw flowcharts
- Write pseudocode
- Create simple decision tables

Practical:

Design solution for:

- Student grading system
- ATM withdrawal system

✅ Week 8: Introduction to Programming (Python Recommended for Level 100)

You can choose either **C** or **Python**.
For beginners, I recommend Python.

If using Python, refer to:

- Python

If using C, refer to:

- C

Topics:

- What is a Programming Language?
- Basic Structure of a Program
- Variables and Data Types
- Input and Output
- Operators
- Simple Conditional Statements

Practical:

Programs:

- Add two numbers
- Find largest of three numbers
- Convert temperature

- Simple grading system

✅ Week 9: Implementation, Testing, Evaluation & Refinement

Topics:

- Debugging
- Testing (manual testing)
- Common programming errors
- Refinement and Optimization
- Good coding practices

Practical:

Mini Project (Individual or Group): Students should:

1. Identify a problem
2. Write algorithm
3. Draw flowchart
4. Write pseudocode
5. Implement in Python or C
6. Test and refine

Example Projects:

- Simple calculator
- Student GPA calculator
- Payroll system
- Result processing system

WEEK ONE

Introduction to Computing and Core Concepts

Introduction to Computing

Computing is the process of using computer systems to solve problems, process data, and produce meaningful results. It involves the use of machines and programs to perform calculations, store information, retrieve data, and automate tasks. Computing is not limited to programming; it includes all activities related to the design, development, and use of computer systems to solve real-world problems.

In today's world, computing plays a vital role in education, business, medicine, banking, communication, engineering, and government. Every time we use an ATM machine, register for courses online, send messages using smartphones, or check examination results, computing is involved. Therefore, understanding computing begins with understanding how computer systems function and how problems are solved using structured approaches.

The Computer System

A computer system is not just a machine. It is a combination of different components working together to perform tasks. A complete computer system consists of hardware, software, and humanware (peopleware).

Hardware

Hardware refers to the physical components of a computer system — the parts that can be seen and touched. Examples include the keyboard, mouse, monitor, system unit, printer, scanner, and storage devices. Hardware performs the physical operations such as receiving input, processing data, and producing output.

The Central Processing Unit (CPU) is often referred to as the "brain" of the computer because it performs calculations and executes instructions. However, without other components, the CPU alone cannot function effectively.

Software

Software refers to the programs and instructions that tell the hardware what to do. Hardware without software is useless because it cannot perform any meaningful task without instructions.

There are different types of software. System software controls the overall operation of the computer. An example is Microsoft Windows, which manages memory, files, and devices. Another example is Linux.

Application software helps users perform specific tasks. For example, Microsoft Word is used for typing documents.

Programming software, such as Visual Studio Code, is used to write computer programs.

Humanware (Peopleware)

Humanware refers to the people who design, operate, manage, and use computer systems. Without human beings, computers cannot function meaningfully. Humans give instructions, interpret results, maintain systems, and make decisions based on computer output.

Examples of humanware include programmers, system analysts, database administrators, computer engineers, and end users. Even a student using a computer for typing assignments is part of the computer system.

Therefore, a computer system is incomplete without humanware. Hardware performs physical operations, software provides instructions, but humanware controls and directs the entire system.

Data and Information

Data refers to raw facts, figures, or symbols that have not yet been processed. Data may consist of numbers, names, dates, or measurements. For example, scores such as 45, 67, and 89 are data. A name like “Amina” is also data.

Information, on the other hand, is processed data that has meaning and usefulness. When the scores 45, 67, and 89 are processed to calculate an average or determine a grade, the result becomes information. Information supports decision-making because it provides understanding and context.

In simple terms, data becomes information after processing. Computers are mainly used to convert data into meaningful information.

The Input–Process–Output (IPO) Model

The Input–Process–Output (IPO) model explains how a computer system works. It describes computing as a cycle of three major activities.

Input is the data entered into the system. This can be done using input devices such as keyboards, mice, scanners, or microphones. For example, typing examination scores into a result system is input.

Process refers to the operations performed on the data. The computer may calculate totals, compare values, sort records, or apply formulas. Processing is done by the CPU according to program instructions.

Output is the result produced after processing. This may appear on a monitor, be printed on paper, or be stored in a file. For example, displaying a student’s GPA is output.

The IPO model is fundamental because every computing task follows this pattern. Whether it is an ATM machine, hospital management system, or mobile banking application, all computing systems follow the Input–Process–Output cycle.

Introduction to Computational Thinking

Computational thinking is a systematic approach to solving problems in a way that a computer can understand. It involves breaking problems into smaller parts, identifying patterns, focusing on important details, and developing step-by-step solutions.

One important aspect of computational thinking is decomposition, which means breaking a large problem into smaller manageable parts. Another aspect is abstraction, which involves focusing only on relevant information while ignoring unnecessary details. Algorithm design is also important; it involves creating clear steps to solve a problem.

Computational thinking helps students develop logical reasoning skills and structured problem-solving abilities. It is not limited to computer science; it can be applied in everyday life and other academic disciplines.

WEEK TWO

Problems and Types of Problems in Computing

In computing, everything begins with a problem. Computers are designed to solve problems, but before a computer can solve any problem, the problem must first be clearly identified and properly understood. Many students think programming is the first step in computing, but that is not correct. The first step is understanding the problem.

A problem can be defined as a situation in which there is a gap between the current state and the desired state, and a solution is needed to bridge that gap. In computing, a problem usually involves processing data to produce meaningful results.

For example, calculating a student's Grade Point Average (GPA), designing a payroll system, managing hospital records, or developing a mobile banking application are all examples of computing problems.

Understanding the nature of problems helps us determine the best method to solve them.

What is a Problem?

A problem exists when there is a task to be accomplished, but the method of achieving it is not immediately obvious. In computing, problems are usually stated in terms of input, required processing, and expected output.

For example, consider the problem: "Calculate the average score of a student." The input is the student's scores.

The process involves adding the scores and dividing by the number of subjects. The output is the average value.

A clearly defined problem makes it easier to design a solution. If a problem is not clearly stated, solving it becomes difficult and confusing.

Characteristics of a Well-Defined Problem

A well-defined problem has the following characteristics:

First, it clearly states what is required. The objective or goal must be specific and understandable.

Second, it specifies the input. We must know what data is available for solving the problem.

Third, it defines the expected output. The result must be clearly described.

Fourth, it includes constraints or limitations. For example, time limits, storage capacity, or specific conditions may affect how the problem is solved.

If these elements are missing, the problem may become ambiguous or difficult to solve effectively.

Routine Problems

Routine problems are problems that follow a known pattern and have a clear method of solution. They usually involve applying a standard formula, rule, or procedure. The steps required to solve routine problems are already known.

Examples of routine problems in computing include:

- Calculating the sum of two numbers
- Finding the largest of three numbers
- Converting Celsius to Fahrenheit
- Calculating simple interest

These problems are predictable and can easily be solved using algorithms. Most beginner programming exercises are routine problems because they help students understand structured thinking.

Routine problems are important because they build foundational skills and logical reasoning.

Non-Routine Problems

Non-routine problems are more complex and do not have an obvious or straightforward solution. They often require creativity, critical thinking, and deeper analysis. The solution steps are not immediately clear.

Examples of non-routine problems include:

- Designing a traffic management system for a busy city
- Developing an artificial intelligence system
- Predicting stock market trends
- Detecting cyber fraud in banking systems

These problems may have multiple possible solutions, and sometimes no perfect solution exists. Solving non-routine problems may require experimentation, research, and advanced problem-solving techniques.

Non-routine problems are common in real-world computing applications.

Well-Defined vs Ill-Defined Problems

A well-defined problem has a clear goal, clear inputs, and clear constraints. The solution path can be logically developed.

An ill-defined problem lacks clarity. The goals may be vague, inputs may not be clearly specified, and constraints may be unknown.

For example:

“Well-defined problem”:

Calculate the total salary of a worker given hourly rate and hours worked.

“Ill-defined problem”:

Improve the education system using technology.

The second example is ill-defined because it is too broad. It does not specify what aspect of education should be improved, what technology should be used, or how success will be measured.

In computing, one of the most important skills is converting ill-defined problems into well-defined ones.

Problem Identification in Computing

Before writing any program, a problem must be carefully analyzed. Problem identification involves asking important questions such as:

- What exactly is the problem?
- Who is affected by the problem?
- What inputs are required?
- What outputs are expected?
- What constraints exist?

If these questions are not answered properly, the final solution may fail even if the program works correctly.

For example, if a programmer develops a student result system without clearly understanding the grading policy, the system may produce incorrect grades even if the calculations are accurate.

Therefore, problem identification is a critical stage in computing.

Importance of Understanding Problem Types

Understanding whether a problem is routine or non-routine helps in selecting the appropriate solution method. Routine problems can often be solved using simple algorithms. Non-routine

problems may require advanced techniques such as abstraction, reduction, or divide-and-conquer strategies, which will be discussed in later weeks.

By learning to classify problems correctly, students develop stronger analytical and logical thinking skills.

WEEK THREE

Introduction to Algorithms & Heuristics

Introduction

In everyday life, people solve problems by following **steps**.

For example:

- Cooking rice
- Brushing your teeth
- Sending an email
- Solving a mathematics problem

In computer science, we also solve problems using **clear steps**.

These steps are called **algorithms**.

Algorithms are very important because **computers follow instructions step by step to perform tasks**.

Meaning of Algorithm

Definition

An **algorithm** is a **clear step-by-step procedure used to solve a problem or complete a task**.

Another simple definition:

An algorithm is a **list of instructions that must be followed in order to solve a problem**.

Simple Example

Problem: Add two numbers.

Algorithm:

1. Start
2. Enter first number (A)
3. Enter second number (B)
4. Add A + B
5. Display the result

6. Stop

This step-by-step process is an **algorithm**.

Characteristics of a Good Algorithm

A good algorithm has some important qualities.

1. Input

An algorithm should accept **data or information**.

Example

- numbers
- names
- scores

Example algorithm input:

Enter student score

2. Output

An algorithm must produce **a result or answer**.

Example:

Display average score

3. Definiteness (Clarity)

Every step must be **clear and easy to understand**.

There should be **no confusion**.

Bad step:

Do something with the numbers

Good step:

Add the two numbers

4. Finiteness

An algorithm must **stop after a limited number of steps**.

Example:

Good algorithm:

1. Start
2. Enter two numbers
3. Add them

4. Display result
5. Stop

Bad algorithm:

Keep adding forever

5. Effectiveness

Each step must be **simple and possible to perform**.

Humans or computers must be able to do the steps easily.

6. Generality

A good algorithm should work for **many inputs**, not only one example.

Example:

Algorithm to add numbers should work for

2 + 3

5 + 10

100 + 250

Examples of Algorithms (Daily Life)

Algorithms are not only for computers.

We use them **every day**.

Example 1: Brushing Teeth

Algorithm:

1. Start
2. Take toothbrush
3. Put toothpaste on brush
4. Brush teeth for 2 minutes
5. Rinse mouth with water
6. Wash toothbrush
7. Stop

Example 2: Making Tea

Algorithm:

1. Start
2. Boil water

3. Add tea leaves or tea bag
4. Add sugar
5. Add milk
6. Stir well
7. Pour into cup
8. Drink tea
9. Stop

Example 3: Logging into a Website

Algorithm:

1. Open browser
2. Enter website address
3. Enter username
4. Enter password
5. Click login
6. If correct → open account
7. If incorrect → show error message
8. Stop

What are Heuristics?

Definition

A **heuristic** is a **simple rule or shortcut used to find a solution quickly**.

It may not always give the **perfect or exact answer**, but it gives a **good or fast solution**.

Heuristics are often used when:

- problems are **very complex**
- exact algorithms take **too long**

Simple Example of Heuristics

Imagine you lost your pen in a classroom.

Algorithm approach

Search every table one by one.

Heuristic approach

Look first where you **usually sit**.

This is faster but **not guaranteed**.

Real Life Heuristic Examples

1. **Choosing shortest route**
    - Use intuition to avoid traffic.
2. **Guessing answer in exam**
    - Eliminate wrong options first.
3. **Finding a book in library**
    - Start with the section where it is most likely located.

Difference Between Algorithm and Heuristic

Feature	**Algorithm**	**Heuristic**
Definition	Exact step-by-step procedure	Shortcut or rule of thumb
Accuracy	Always gives correct result	May not always give correct result
Speed	Sometimes slower	Usually faster
Complexity	More detailed steps	Simpler rules
Example	Sorting numbers using steps	Guessing the largest number quickly

Simple Illustration

Algorithm method

Find largest number in list:

1. Compare number 1 and 2
2. Keep the bigger number
3. Compare with next number
4. Repeat until list ends

Heuristic method

Just look quickly and guess the biggest number.

WEEK FOUR

Introduction

In computer science, algorithms are used to solve different kinds of problems. An algorithm provides a clear set of instructions that a computer can follow in order to produce a solution. However, not every problem in computing can be solved by an algorithm. Some problems have clear procedures that always lead to the correct answer, while others cannot be solved by any algorithm regardless of how powerful the computer is.

Because of this, computer scientists study the nature of problems in order to determine whether they are solvable or unsolvable. Understanding these concepts helps programmers know the limits of computation and also helps them design better and more efficient algorithms.

What Makes a Problem Solvable?

A problem is considered solvable if there exists an algorithm that can always produce the correct answer for every valid input in a finite number of steps. This means that the algorithm must start, perform a series of well-defined operations, and eventually stop after producing a result. If such an algorithm exists, the problem is said to be solvable by a computer.

For example, the problem of adding two numbers is solvable because we can design a simple algorithm that takes the numbers as input, adds them together, and displays the result. Similarly, finding the largest number in a list is also solvable because we can compare the numbers step by step until the largest one is found. In both cases, the algorithm always gives the correct answer and eventually stops.

Unsolvable Problems

Although many problems can be solved using algorithms, there are also problems that cannot be solved by any algorithm. These are called unsolvable problems. An unsolvable problem is one for which no algorithm can always provide the correct answer for every possible input.

This limitation does not occur because computers are slow or lack memory. Instead, it happens because of fundamental mathematical limits in computation. Even if computers become faster and more powerful in the future, these types of problems will still remain unsolvable.

Understanding unsolvable problems is important because it helps computer scientists avoid wasting time trying to create algorithms that are mathematically impossible.

Decidability

In theoretical computer science, the concept of decidability is used to classify problems. A problem is said to be decidable if there exists an algorithm that can always give a definite “yes” or “no” answer for every input.

For example, determining whether a number is even or odd is a decidable problem. An algorithm can simply divide the number by two and check the remainder. If the remainder is zero, the number is even; otherwise, it is odd. This process always produces a correct answer and stops after a small number of steps.

On the other hand, a problem is undecidable if no algorithm can be created to always produce a correct yes-or-no answer. Such problems belong to the category of unsolvable problems.

The Halting Problem (Conceptual Explanation)

One of the most famous examples of an undecidable problem is the **Halting Problem**, which was introduced by the British mathematician and computer scientist **Alan Turing**.

The Halting Problem asks whether it is possible to create a program that can examine another computer program and determine whether that program will eventually stop running or continue running forever. In simple terms, the question is whether a computer can always predict the behavior of another program.

After careful mathematical analysis, Turing proved that it is impossible to design such a universal program. In other words, there is no algorithm that can always determine whether every possible program will halt or run forever. Because of this, the Halting Problem is considered an undecidable problem.

Computational Limits

Although computers are very powerful machines, they still have certain limitations. These limitations are known as computational limits. Some problems cannot be solved because they require too much time, too much memory, or because they are mathematically impossible to solve using algorithms.

For instance, some algorithms may require an extremely large number of steps to produce an answer. If the number of steps is too large, the algorithm may take many years or even centuries to finish. In such cases, the problem may be theoretically solvable but practically impossible to solve within a reasonable time.

Understanding computational limits helps computer scientists focus on problems that can be solved efficiently and avoid problems that cannot realistically be solved.

Efficiency of Algorithms (Time and Space)

Even when a problem is solvable, there may be many different algorithms that can solve it. Some algorithms may take longer to run, while others may require more memory. Therefore, computer scientists study the efficiency of algorithms in order to determine which one performs better.

Algorithm efficiency is usually measured in two main ways: time and space. Time efficiency refers to the amount of time an algorithm takes to complete its task. Space efficiency refers to the amount of memory required by the algorithm while it is running.

For example, suppose two algorithms solve the same problem. If one algorithm completes the task in a few seconds while the other takes several minutes, the first algorithm is considered more time-efficient. Similarly, if one algorithm uses very little memory while another uses a large amount of memory, the first algorithm is considered more space-efficient.

Studying time and space efficiency helps programmers develop algorithms that perform well even when dealing with large amounts of data.

In summary, not all problems in computing can be solved using algorithms. A problem is considered solvable if there exists an algorithm that always produces the correct answer and eventually stops. Some problems, however, are unsolvable because no algorithm can be created to solve them for all possible cases.

The concept of decidability helps classify problems into decidable and undecidable categories. The Halting Problem is a well-known example of an undecidable problem and demonstrates the limits of computation. Finally, even when a problem is solvable, the efficiency of the algorithm must be considered in terms of the time it takes to run and the amount of memory it uses.

Understanding these ideas helps computer scientists design better algorithms and also recognize the fundamental limits of what computers can achieve.